Министерство цифрового развития, связи и массовых коммуникаций
Ордена Трудового Красного Знамени
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра «Математическая Кибернетика и Информационные технологии»

Лабораторная работа №3

Многотабличные запросы и аналитические представления

Выполнил: Студент группы
БВТ2503
Данилов Всеволод Александрович

Преподаватель: Юлия Александровна Полуян

Москва 2026

Цели работы:

Научиться составлять многотабличные и аналитические запросы к базе данных маркетплейса, освоить соединения таблиц (INNER JOIN, LEFT JOIN), группировку и агрегацию с условием HAVING, скалярные и коррелированные подзапросы, общие табличные выражения (CTE, включая рекурсивный обход иерархии), а также создание и использование аналитических представлений (VIEW).

Техническое задание:

1. Напишите запрос с внутренним соединением не менее трёх таблиц (товар — категория — продавец).

2. Напишите запрос с LEFT JOIN, показывающий строки без соответствий в связанной таблице (например, товары без отзывов).

3. Постройте запрос с группировкой и агрегацией: по каждой категории — количество товаров и средняя цена.

4. Примените HAVING для фильтрации групп (например, продавцы с выручкой выше заданной).

5. Напишите по одному запросу с подзапросом каждого вида: скалярным, с IN и с EXISTS (или коррелированным).

6. Используйте CTE для разбиения сложного запроса на шаги; реализуйте рекурсивный CTE для обхода дерева категорий.

7. Создайте не менее двух представлений для типовых отчётов и продемонстрируйте обращение к ним.

Ход выполнения работы:

1. Запрос с внутренним соединением не менее трёх таблиц (товар — категория — продавец)

Назначение: получение подробного каталога активных товаров с указанием названия товара, товарной категории, магазина-продавца, контактного email и розничной цены (соединение products, categories, sellers, users).

Query | Query History

```sql
SELECT
    p.id           AS product_id,
    p.title        AS product_title,
    c.name         AS category_name,
    s.store_name   AS seller_store,
    u.email        AS seller_email,
    p.price,
    p.is_active
FROM products AS p
JOIN categories AS c ON c.id = p.category_id
JOIN sellers    AS s ON s.id = p.seller_id
JOIN users      AS u ON u.id = s.user_id
WHERE p.is_active = TRUE
ORDER BY p.price DESC
LIMIT 10;
```

	product_id bigint	product_title text	category_name text	seller_store text	seller_email text	price numeric (12,2)	is_active boolean
1	2	Смартфон iPhone 15	Смартфоны	ТехноМир	sales@technomir....	79990.00	true
2	3	Ноутбук Lenovo IdeaPad	Ноутбуки	ТехноМир	sales@technomir....	54990.00	true
3	10	Смартфон Xiaomi 13T	Смартфоны	ТехноМир	sales@technomir....	42990.00	true
4	1	Смартфон Galaxy A54	Смартфоны	ТехноМир	sales@technomir....	34990.00	true
5	4	Наушники Sony WH-1000X...	Наушники	ТехноМир	sales@technomir....	29990.00	true
6	8	Велосипед горный Stels	Велоспорт	СпортЛайн	sport@shop.ru	18900.00	true
7	647	Товар №647	Фитнес	Книголюб	info@knigolub.ru	9982.84	true
8	556	Товар №556	Учебники	ТехноМир	sales@technomir....	9979.60	true
9	720	Товар №720	Художественная литерату...	СпортЛайн	sport@shop.ru	9975.49	true
10	277	Товар №277	Велоспорт	СпортЛайн	sport@shop.ru	9967.04	true

2. Запрос с LEFT JOIN, показывающий строки без соответствий (Anti-Join)

Назначение: поиск активных товаров, на которые покупатели еще не оставили ни одного отзыва (LEFT JOIN ... WHERE r.id IS NULL) для запуска маркетинговых акций.

Query | Query History

```sql
SELECT
    p.id            AS product_id,
    p.title         AS product_title,
    c.name          AS category_name,
    p.price,
    r.id            AS review_id
FROM products AS p
JOIN categories AS c ON c.id = p.category_id
LEFT JOIN reviews AS r ON r.product_id = p.id
WHERE r.id IS NULL
ORDER BY p.id ASC
LIMIT 10;
```

	product_id bigint	product_title text	category_name text	price numeric (12,2)	review_id bigint
1	1	Смартфон Galaxy A54	Смартфоны	34990.00	[null]
2	2	Смартфон iPhone 15	Смартфоны	79990.00	[null]
3	3	Ноутбук Lenovo IdeaPad	Ноутбуки	54990.00	[null]
4	4	Наушники Sony WH-1000X...	Наушники	29990.00	[null]
5	5	Роман «Мастер и Маргарит...	Художественная литерату...	890.00	[null]
6	6	Учебник «Базы данных»	Учебники	2500.00	[null]
7	7	Гантели разборные 2×10 кг	Фитнес	4500.00	[null]
8	8	Велосипед горный Stels	Велоспорт	18900.00	[null]
9	9	Наушники AirPods Pro 2	Наушники	24990.00	[null]
10	10	Смартфон Xiaomi 13T	Смартфоны	42990.00	[null]

3. Запрос с группировкой и агрегацией по категориям (GROUP BY)

Назначение: комплексный аналитический срез ассортимента маркетплейса: количество товаров, средняя, минимальная и максимальная цена по каждой категории.

```sql
SELECT
    c.id                                AS category_id,
    c.name                              AS category_name,
    COUNT(p.id)                         AS products_count,
    ROUND(COALESCE(AVG(p.price), 0), 2) AS avg_price,
    COALESCE(MIN(p.price), 0)           AS min_price,
    COALESCE(MAX(p.price), 0)           AS max_price
FROM categories AS c
LEFT JOIN products AS p ON p.category_id = c.id AND p.is_active = TRUE
GROUP BY c.id, c.name
ORDER BY products_count DESC, c.name ASC;
```

	category_id bigint	category_name text	products_count bigint	avg_price numeric	min_price numeric	max_price numeric
1	4	Смартфоны	117	6785.08	1009.90	79990.00
2	3	Спорт	115	5387.42	1068.05	9849.04
3	5	Ноутбуки	103	5974.51	1071.76	54990.00
4	6	Наушники	96	6001.61	1033.42	29990.00
5	7	Художественная литерату…	94	5712.74	890.00	9975.49
6	2	Книги	91	5196.87	1007.39	9895.19
7	8	Учебники	86	5709.97	1019.08	9979.60
8	9	Фитнес	85	5442.80	1052.85	9982.84
9	1	Электроника	53	5312.23	1033.80	9897.96
10	10	Велоспорт	36	6007.11	1065.27	18900.00

4. Применение HAVING для фильтрации сгруппированных данных

Назначение: отбор ведущих магазинов-продавцов, суммарная выручка которых по всем неотмененным заказам превышает 3 000 000 рублей.

```sql
SELECT
    s.id                                        AS seller_id,
    s.store_name,
    COUNT(DISTINCT o.id)                        AS orders_count,
    SUM(oi.quantity)                            AS total_items_sold,
    ROUND(SUM(oi.quantity * oi.unit_price), 2)  AS total_revenue
FROM sellers AS s
JOIN products AS p     ON p.seller_id = s.id
JOIN order_items AS oi ON oi.product_id = p.id
JOIN orders AS o       ON o.id = oi.order_id
WHERE o.status != 'cancelled'
GROUP BY s.id, s.store_name
HAVING SUM(oi.quantity * oi.unit_price) > 3000000.00
ORDER BY total_revenue DESC;
```

	seller_id bigint	store_name text	orders_count bigint	total_items_sold bigint	total_revenue numeric
1	2	Книголюб	442	1115	6177925.09
2	1	ТехноМир	442	1084	6093309.07

5.1. Скалярный подзапрос в условиях фильтрации

Назначение: выборка товаров, цена которых превышает среднюю цену активных товаров по всей базе маркетплейса, с расчетом разницы со средней ценой.

```sql
SELECT
    p.id                                                                    AS product_id,
    p.title                                                                 AS product_title,
    p.price,
    ROUND((SELECT AVG(price) FROM products WHERE is_active = TRUE), 2) AS market_avg_price,
    ROUND(p.price - (SELECT AVG(price) FROM products WHERE is_active = TRUE), 2) AS diff_from_avg
FROM products AS p
WHERE p.is_active = TRUE
  AND p.price > (SELECT AVG(price) FROM products WHERE is_active = TRUE)
ORDER BY p.price DESC
LIMIT 10;
```

	product_id bigint	product_title text	price numeric (12,2)	market_avg_price numeric	diff_from_avg numeric
1	2	Смартфон iPhone 15	79990.00	5783.50	74206.50
2	3	Ноутбук Lenovo IdeaPad	54990.00	5783.50	49206.50
3	10	Смартфон Xiaomi 13T	42990.00	5783.50	37206.50
4	1	Смартфон Galaxy A54	34990.00	5783.50	29206.50
5	4	Наушники Sony WH-1000X...	29990.00	5783.50	24206.50
6	8	Велосипед горный Stels	18900.00	5783.50	13116.50
7	647	Товар №647	9982.84	5783.50	4199.34
8	556	Товар №556	9979.60	5783.50	4196.10
9	720	Товар №720	9975.49	5783.50	4191.99
10	277	Товар №277	9967.04	5783.50	4183.54

5.2. Подзапрос с предикатом IN

Назначение: поиск клиентов (users), которые оформляли заказы на цифровую электронику из категорий «Смартфоны» и «Ноутбуки».

```sql
SELECT
    u.id        AS user_id,
    u.full_name,
    u.email,
    u.phone
FROM users AS u
WHERE u.id IN (
    SELECT o.user_id
    FROM orders AS o
    JOIN order_items AS oi ON oi.order_id = o.id
    JOIN products AS p     ON p.id = oi.product_id
    WHERE p.category_id IN (
        SELECT c.id
        FROM categories AS c
        WHERE c.name IN ('Смартфоны', 'Ноутбуки')
    )
)
ORDER BY u.id ASC;
```

	user_id bigint	full_name text	email text	phone text
1	1	Иван Петров	ivan@mail.ru	+790011122…
2	2	Мария Сидоро…	maria@mail.ru	+790044455…
3	3	ООО ТехноМир	sales@technomir.…	+780020030…
4	4	ООО Книголюб	info@knigolub.ru	+780050060…
5	5	Алексей Смир…	alex@mail.ru	+790077788…
6	6	ООО СпортЛайн	sport@shop.ru	+780090010…
7	7	Елена Кузнецо…	elena@mail.ru	[null]
8	8	Дмитрий Волк…	dmitry@mail.ru	+790033344…

5.3. Подзапрос с предикатом EXISTS (коррелированный)

Назначение: поиск продавцов, у которых есть хотя бы один товар с нулевым остатком на складе (stock.quantity = 0).

```sql
SELECT
    s.id          AS seller_id,
    s.store_name,
    s.description
FROM sellers AS s
WHERE EXISTS (
    SELECT 1
    FROM products AS p
    JOIN stock AS st ON st.product_id = p.id
    WHERE p.seller_id = s.id
      AND st.quantity = 0
);
```

	seller_id bigint	store_name text	description text
1	1	ТехноМир	Электроника и гадже…

6.1. Аналитический CTE (доли рынка продавцов)

Назначение: пошаговый расчет оборота продавцов и вычисление их доли рынка (market share %) в суммарной выручке площадки.

```sql
WITH seller_revenue AS (
    SELECT
        s.id                                        AS seller_id,
        s.store_name,
        COUNT(DISTINCT o.id)                        AS orders_count,
        COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS revenue
    FROM sellers AS s
    JOIN products AS p     ON p.seller_id = s.id
    JOIN order_items AS oi ON oi.product_id = p.id
    JOIN orders AS o       ON o.id = oi.order_id
    WHERE o.status != 'cancelled'
    GROUP BY s.id, s.store_name
),
total_market AS (
    SELECT SUM(revenue) AS total_revenue
    FROM seller_revenue
)
SELECT
    sr.seller_id,
    sr.store_name,
    sr.orders_count,
    ROUND(sr.revenue, 2) AS revenue,
    ROUND((sr.revenue / NULLIF(tm.total_revenue, 0)) * 100, 2) AS market_share_pct
FROM seller_revenue AS sr
CROSS JOIN total_market AS tm
ORDER BY sr.revenue DESC;
```

	seller_id bigint	store_name text	orders_count bigint	revenue numeric	market_share_pct numeric
1	2	Книголюб	442	6177925.09	50.34
2	1	ТехноМир	442	6093309.07	49.66

6.2. Рекурсивный CTE (обход дерева категорий каталога)

Назначение: рекурсивный обход иерархии категорий (categories.parent_id) сверху вниз с расчетом уровня вложенности (level) и формированием полного пути (full_path).

```sql
WITH RECURSIVE category_hierarchy AS (
    SELECT
        c.id,
        c.parent_id,
        c.name,
        1 AS level,
        c.name::TEXT AS full_path
    FROM categories AS c
    WHERE c.parent_id IS NULL

    UNION ALL

    SELECT
        c.id,
        c.parent_id,
        c.name,
        ch.level + 1 AS level,
        (ch.full_path || ' -> ' || c.name)::TEXT AS full_path
    FROM categories AS c
    JOIN category_hierarchy AS ch ON ch.id = c.parent_id
)
SELECT
    id AS category_id,
    REPEAT('    ', level - 1) || '• ' || name AS visual_tree,
    level,
    full_path
FROM category_hierarchy
ORDER BY full_path;
```

	category_id bigint	visual_tree text	level integer	full_path text
1	2	• Книги	1	Книги
2	8	• Учебники	2	Книги -> Учебники
3	7	• Художественная литерату...	2	Книги -> Художественная литерату...
4	3	• Спорт	1	Спорт
5	10	• Велоспорт	2	Спорт -> Велоспорт
6	9	• Фитнес	2	Спорт -> Фитнес
7	1	• Электроника	1	Электроника
8	6	• Наушники	2	Электроника -> Наушники
9	5	• Ноутбуки	2	Электроника -> Ноутбуки
10	4	• Смартфоны	2	Электроника -> Смартфоны

7.1. Представление v_product_catalog_details (витрина каталога с рейтингом и остатком)

Назначение: постоянная витрина товаров, объединяющая данные о товарах, категориях, продавцах, складских запасах и среднем рейтинге отзывов.

```sql
CREATE OR REPLACE VIEW v_product_catalog_details AS
SELECT
    p.id                                AS product_id,
    p.title                             AS product_name,
    c.name                              AS category_name,
    s.store_name                        AS seller_name,
    p.price,
    p.is_active,
    COALESCE(st.quantity, 0)            AS stock_quantity,
    COUNT(r.id)                         AS reviews_count,
    ROUND(COALESCE(AVG(r.score), 0), 1) AS avg_rating
FROM products AS p
JOIN categories AS c ON c.id = p.category_id
JOIN sellers AS s    ON s.id = p.seller_id
LEFT JOIN stock AS st ON st.product_id = p.id
LEFT JOIN reviews AS r ON r.product_id = p.id
GROUP BY p.id, p.title, c.name, s.store_name, p.price, p.is_active, st.quantity;

SELECT
    product_id, product_name, category_name, seller_name, price, stock_quantity, avg_rating
FROM v_product_catalog_details
WHERE is_active = TRUE AND stock_quantity > 0
ORDER BY avg_rating DESC, price DESC
LIMIT 10;
```

	product_id bigint	product_name text	category_name text	seller_name text	price numeric (12,2)	stock_quantity integer	avg_rating numeric
1	2	Смартфон iPhone 15	Смартфоны	ТехноМир	79990.00	20	0.0
2	3	Ноутбук Lenovo IdeaPad	Ноутбуки	ТехноМир	54990.00	15	0.0
3	10	Смартфон Xiaomi 13T	Смартфоны	ТехноМир	42990.00	35	0.0
4	1	Смартфон Galaxy A54	Смартфоны	ТехноМир	34990.00	50	0.0
5	4	Наушники Sony WH-1000X...	Наушники	ТехноМир	29990.00	30	0.0
6	8	Велосипед горный Stels	Велоспорт	СпортЛайн	18900.00	10	0.0
7	7	Гантели разборные 2×10 кг	Фитнес	СпортЛайн	4500.00	25	0.0
8	6	Учебник «Базы данных»	Учебники	Книголюб	2500.00	40	0.0
9	5	Роман «Мастер и Маргарит...	Художественная литерату...	Книголюб	890.00	100	0.0

7.2. Представление v_seller_performance_summary (управленческий дашборд продаж)

Назначение: аналитическая витрина для контроля ключевых показателей эффективности (KPI) магазинов: ассортимент, число выполненных заказов, проданные единицы, выручка и средний чек.

```sql
CREATE OR REPLACE VIEW v_seller_performance_summary AS
SELECT
    s.id                                                    AS seller_id,
    s.store_name,
    COUNT(DISTINCT p.id)                                    AS total_products,
    COUNT(DISTINCT o.id)                                    AS completed_orders_count,
    COALESCE(SUM(oi.quantity), 0)                           AS total_units_sold,
    ROUND(COALESCE(SUM(oi.quantity * oi.unit_price), 0), 2)  AS total_revenue,
    ROUND(COALESCE(SUM(oi.quantity * oi.unit_price) / NULLIF(COUNT(DISTINCT o.id), 0), 0), 2) AS avg_order_value
FROM sellers AS s
LEFT JOIN products AS p     ON p.seller_id = s.id
LEFT JOIN order_items AS oi ON oi.product_id = p.id
LEFT JOIN orders AS o       ON o.id = oi.order_id AND o.status IN ('paid', 'shipped', 'delivered')
GROUP BY s.id, s.store_name;

SELECT
    seller_id, store_name, total_products, completed_orders_count, total_units_sold, total_revenue, avg_order_value
FROM v_seller_performance_summary
ORDER BY total_revenue DESC;
```

	seller_id bigint	store_name text	total_products bigint	completed_orders_count bigint	total_units_sold bigint	total_revenue numeric	avg_order_value numeric
1	2	Книголюб	547	369	1265	7076879.75	19178.54
2	1	ТехноМир	278	369	1232	6956136.85	18851.32
3	3	СпортЛайн	275	0	0	0.00	0.00

Код можно посмотреть по ссылке:
https://github.com/carsonxman/sql-labs/blob/main/lab03/lab03sql.sql

Ответы на контрольные вопросы:

1. Что такое соединение таблиц и по какому условию оно обычно выполняется?

Соединение таблиц (JOIN) — это базовая операция реляционной алгебры, объединяющая строки из двух или более таблиц в единый результирующий набор на основе заданного логического условия (секция ON). В нормализованных реляционных базах данных соединение чаще всего выполняется по условию эквивалентности (Equi-Join) — равенству значения первичного ключа (Primary Key) родительской таблицы и внешнего ключа (Foreign Key) дочерней таблицы (например, ON c.id = p.category_id). Также возможно соединение по неравенствам или диапазонам (Theta-Join), а при совпадении имён связующих столбцов допустима краткая форма USING (имя_столбца).

2. Чем INNER JOIN отличается от LEFT JOIN? Приведите пример, где нужен именно LEFT JOIN.

INNER JOIN (внутреннее соединение) оставляет в выборке исключительно те строки, для которых условие соединения истинно в обеих таблицах; строки, не нашедшие соответствия, отбрасываются. LEFT JOIN (левое внешнее соединение) гарантированно сохраняет абсолютно все строки левой таблицы, а в столбцы правой таблицы подставляет значения NULL, если соответствие не найдено. Пример необходимости LEFT JOIN: получение полного перечня категорий с количеством товаров в каждой из них (включая категории с 0 товаров), либо поиск сущностей без связанных данных (паттерн Anti-Join: выборка товаров, у которых ещё нет отзывов, через LEFT JOIN reviews r ON r.product_id = p.id WHERE r.id IS NULL).

3. Какие агрегатные функции вы знаете? В чём различие между COUNT(*) и COUNT(столбец)?

К основным стандартным агрегатным функциям относятся: COUNT (подсчёт числа строк), SUM (суммирование), AVG (вычисление среднего арифметического), MIN и MAX (нахождение минимального и максимального значений). Различие заключается в том, что COUNT(*) подсчитывает абсолютно все физические строки в группе или таблице, включая строки, содержащие NULL-значения. В свою очередь, COUNT(столбец) подсчитывает только те строки, в которых значение указанного столбца строго НЕ равно NULL. Все остальные агрегатные функции (SUM, AVG, MIN, MAX) также всегда автоматически игнорируют значения NULL.

4. Для чего предназначена секция GROUP BY и что допустимо в списке выборки при группировке?

Секция GROUP BY предназначена для объединения строк с одинаковыми значениями в указанных столбцах в группы для последующего вычисления сводных показателей агрегатными функциями. В списке выборки (SELECT) при использовании группировки допустимо указывать только: 1) столбцы, явно перечисленные в секции GROUP BY (или функционально от них зависящие); 2) агрегатные функции над любыми столбцами (SUM, AVG, COUNT, MIN, MAX); 3) константы и детерминированные выражения. Указание неагрегированных столбцов, отсутствующих в GROUP BY, является синтаксической ошибкой, так как СУБД не может однозначно сопоставить множество значений одной группы единственной итоговой строке.

5. В чём различие между WHERE и HAVING и в каком порядке они выполняются?

Секции WHERE и HAVING выполняют фильтрацию данных на разных этапах конвейера обработки запроса: 1) WHERE фильтрует индивидуальные исходные строки ДО выполнения группировки (в секции WHERE категорически запрещено использовать агрегатные функции); 2) HAVING фильтрует уже сформированные группы ПОСЛЕ выполнения группировки и вычисления агрегатных функций (например, HAVING SUM(oi.quantity * oi.unit_price) > 3000000). Порядок логического выполнения запроса СУБД: FROM -> JOIN (ON) -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT / OFFSET.

6. Что такое подзапрос? Чем коррелированный подзапрос отличается от некоррелированного?

Подзапрос — это оператор SELECT, вложенный внутрь другого объемлющего SQL-запроса (в секции SELECT, FROM, WHERE или HAVING). Некоррелированный подзапрос автономен: он не зависит от внешнего запроса, выполняется оптимизатором СУБД ровно один раз перед главным запросом, а его результат подставляется во внешний запрос. Коррелированный подзапрос ссылается на столбцы из текущей строки внешнего запроса (например, WHERE p.seller_id = s.id). Такой подзапрос логически вычисляется заново для каждой отдельной строки, обрабатываемой внешним запросом.

7. Как работает предикат EXISTS и когда его удобно применять?

Предикат EXISTS принимает в качестве аргумента подзапрос и возвращает значение TRUE, если подзапрос вернул хотя бы одну строку (независимо от значений столбцов, поэтому общепринято писать SELECT 1), и FALSE, если результат подзапроса пуст. Предикат EXISTS оптимизирован для быстрой проверки

наличия связей, так как СУБД прекращает сканирование строк подзапроса сразу же при нахождении первого соответствия (Short-Circuit Evaluation). Его удобно применять для проверки факта существования связанных записей (например, наличие у продавца товаров с нулевым остатком), а NOT EXISTS — для поиска отсутствия связей без риска ошибок, связанных со значениями NULL в предикате NOT IN.

8. Что такое общее табличное выражение (CTE) и для чего нужен рекурсивный CTE?

Общее табличное выражение (CTE, конструкция WITH) — это именованный временный набор данных, определяемый в начале запроса и доступный на протяжении его выполнения. CTE позволяет разбить сложный запрос на логические читаемые шаги и избежать дублирования вложенных подзапросов. Рекурсивный CTE (WITH RECURSIVE) необходим для обхода и обработки данных с древовидной, иерархической или графовой структурой (например, каталог категорий и подкатегорий маркетплейса с самоссылкой parent_id). Он состоит из анкерной части (выборка корневых элементов) и рекурсивной части, которая итеративно присоединяет дочерние узлы через UNION ALL до исчерпания связей.

9. Что такое представление (VIEW)? Хранит ли обычное представление данные?

Представление (VIEW) — это сохранённый в словаре базы данных именованный SQL-запрос, к которому можно обращаться как к виртуальной таблице. Обычное представление физически НЕ хранит данные на диске. Каждый раз при обращении к представлению СУБД выполняет базовый SQL-запрос и возвращает актуальные данные из исходных таблиц. Представления служат для упрощения сложных многотабличных выборок, создания аналитических витрин и разграничения прав доступа к отдельным столбцам таблиц.

10. Чем материализованное представление отличается от обычного?

В отличие от обычного виртуального представления, материализованное представление (MATERIALIZED VIEW) физически сохраняет результат выполнения запроса на диске в виде реальной таблицы. Это обеспечивает мгновенное считывание данных и поддержку вторичных индексов для ускорения тяжёлых аналитических агрегатов над миллионами строк. Однако данные в материализованном представлении статичны и не изменяются автоматически при изменении базовых таблиц — для их актуализации требуется периодический вызов

команды REFRESH MATERIALIZED VIEW (в том числе с параметром CONCURRENTLY для работы без блокировки читателей).

Вывод:

В ходе выполнения ЛР №3 освоены практические приёмы составления многотабличных и аналитических запросов в СУБД PostgreSQL. Реализованы внутренние (INNER JOIN) и левые внешние (LEFT JOIN) соединения, включая шаблон Anti-Join для поиска товаров без отзывов. Освоена группировка с агрегатными функциями (GROUP BY) и фильтрацией групп через секцию HAVING по суммарной выручке. Написаны и протестированы скалярные подзапросы, подзапросы с предикатом IN и коррелированные подзапросы с предикатом EXISTS. С помощью CTE (Common Table Expressions) реализован многошаговый аналитический расчет доли рынка продавцов, а с помощью рекурсивного CTE (WITH RECURSIVE) — полный обход и визуализация иерархического дерева категорий маркетплейса. Спроектированы постоянные аналитические представления (VIEW) для витрины каталога и управленческого дашборда эффективности продавцов.